# Information-Theoretic Signatures of Human and AI-Generated Software

Ertugrul Omer Gul 480426660

As generative artificial intelligence becomes prevalent in software, the challenges it brings become more apparent. The reduced cost of generating software changes presents opportunities and challenges for open-source software communities where code contributions must ultimately be reviewed and maintained by human developers. The lowered cost of code production and the rising cost of its review and maintenance is a point of concern in these communities as well as in current literature. Afroz et al. coin the term "*AI-DDoS"* which refers to a flood of plausible-looking, low-quality GenAI contributions that disrupts the contribution process and overwhelms the limited time and resources maintainers have to review contributions (Afroz, 2026). Similarly, an analysis of 281 open-source repository contribution policies by Hora et al. shows a common push for human involvement and disclosure of AI assistance as requirements (Hora, 2026). Major open-source projects have also introduced policies emphasising that AI-generated changes must be understood, tested, and ultimately owned by human contributors.

In such a world, the differentiability of AI-augmented development from human-written software becomes key. Even if open-source repositories introduce requirements pertaining to the "AI-ness" of contributions, unless they have the tools to enforce such requirements they are, in a sense, meaningless. As Frank Herbert put in his famous novel, Dune, *"The power to destroy a thing is the absolute control over it"* (Herbert, 1965). In a less dramatic tone, unless we have the means to recognise something is AI, we can have no meaningful options as to how we deal with it. So then, perhaps; *"The power to recognise AI is the power to control it"*.

Recent work on distinguishing human and AI-generated source code has primarily approached the problem as one of authorship detection. Approaches include classifiers based on pre-trained models such as CodeBERT and UniXcoder (Nguyen et al., 2024; Xu et al., 2025), fine-tuned language models and classifiers using static code metrics or Abstract Syntax Tree (AST) representations (Suh et al., 2025), and hand-crafted stylometric features

capturing characteristics such as naming conventions, indentation, commenting behaviour and code structure (Park et al., 2025).

Furthermore, other work investigates statistical properties such as lexical diversity, code naturalness, likelihood and perturbation-based measures for detecting machine-generated code (Shi et al., 2025). These studies demonstrate that human and AI-generated code contain measurable signals that can support authorship detection, although recent evaluations indicate that detector performance varies substantially across models and programming languages (Orel et al., 2026). Information-theoretic measures have separately been shown to characterise source code through quantities such as token and AST-based entropy (Torres et al., 2025). This project therefore investigates whether information-theoretic characteristics of source-code structure can provide additional, interpretable signals for distinguishing human and AI-generated software changes.

Our central research question is as follows: *Do human-written and AI-generated solutions to the same software engineering tasks exhibit systematically different information-theoretic structure?* We hypothesise that AI-generated patches will exhibit different levels of entropy and sequential dependence from human-written patches. More formally:

*H1: AI-generated patches will differ from human patches in token entropy.*

*H2: AI-generated patches will exhibit greater short-range sequential dependence and predictability than matched human patches, reflected in higher short-range MI and AIS.*

In particular, we expect that AI-generated code may be more locally regular and predictable due to the autoregressive process through which LLMs generate code.

The dataset chosen for this study is SWE-bench (Jimenez et al., 2024). In the dataset, each instance contains a real GitHub issue together with the software patch that resolved it. Publicly released SWE-bench Verified patches generated by Claude Opus 4.5, Gemini 3 Flash, and GPT-5.2-Codex will be paired with the corresponding human solutions.

The analysis will focus on the source code introduced by each patch. Metadata such as file paths and unchanged lines of code will not be addressed. Only additions or replacements of lines of code will be taken into account. The code under analysis will be tokenised into a finite vocabulary of syntactic categories such as identifiers, keywords, operators, literals and

delimiters. As opposed to per-character encoding, this approach is expected to be less affected by superficial choices of identifier names, whitespace and formatting. Syntactic tokenisation instead reduces the vocabulary size and allows us to focus more directly on the code structure.

The methodology used to address our research questions will include Shannon entropy, mutual information (MI) and active information storage (AIS) measures. Here we treat source code as a structured symbolic sequence in which local and longer-range dependencies emerge from the interaction of syntax and programming conventions. Where $X$ is the corresponding discrete random variable over the fixed token vocabulary and $X_t$ is the token at sequence position t, The Shannon entropy,

$$H(X),$$

will measure uncertainty in the distribution of token categories in the code patch. Simply put a lower entropy would indicate that the sequence is concentrated around a smaller or less evenly distributed set of token types, while higher entropy would indicate greater token diversity. However, entropy ignores token ordering. Thus, secondly, we look at the Mutual information,

$$I(X_t; X_{t+k}),$$

which will quantify statistical dependence between tokens separated by $k$ positions. This will allow us to measure whether the rate at which this dependence decays with token distance differs between the two groups. Lastly the active information storage will allow us to measure the extent to which the recent history of a token sequence provides information about the next token:

$$A_X(k) = I(X_n^k; X_{n+1}),$$

where $X_n^k$ denotes the previous $k$ token categories and $X_{n+1}$ the subsequent token. This will provide us a measure of sequential predictability that cannot be obtained from Shannon entropy alone. Several small history lengths will be tested for AIS and checked for stability. For each of the measures we will compare each model to the corresponding human solutions

with the goal of testing our hypothesis. For example;

$$A_X^{Claude} \; compared \; to \; A_X^{Human}$$

$$A_X^{Gemini} compared \; to \; A_X^{Human}$$

$$A_X^{GPT} compared \; to \; A_X^{Human}.$$

Thus, entropy captures token diversity, mutual information captures pairwise sequential dependence, and AIS captures history-based predictability. As estimates, particularly MI and AIS, may be unreliable for short patches, patches below a predefined minimum token count will be excluded and sensitivity to this threshold will be examined. JIDT's discrete estimators will be used, with human and AI measurements compared as matched pairs within each SWE-bench task.

# References


Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. R. (2024). SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*.

Afroz, S., Miller, C., Menezes, T., Gilmour, E., Sarma, A., & Feng, Z. (2026). *"AI slop is DDoSing open source": Understanding the impact of AI-generated contributions on open source sustainability*. arXiv. https://arxiv.org/abs/2607.04003

Hora, A., Robbes, R., & Zacchiroli, S. (2026). *"We permit the use of AI, but [...]": The landscape of AI policies in popular open source projects*. arXiv. https://arxiv.org/abs/2609.07542

Nguyen, P. T., Di Rocco, J., Di Sipio, C., Rubei, R., Di Ruscio, D., & Di Penta, M. (2024). GPTSniffer: A CodeBERT-based classifier to detect source code written by ChatGPT. *Journal of Systems and Software, 214*, 112059. https://doi.org/10.1016/j.jss.2024.112059

Xu, X., Ni, C., Guo, X., Liu, S., Wang, X., Liu, K., & Yang, X. (2025). Distinguishing LLM-generated from human-written code by contrastive learning. *ACM Transactions on Software Engineering and Methodology, 34*(4), Article 91, 1–31. https://doi.org/10.1145/3705300

Suh, H., Tafreshipour, M., Li, J., Bhattiprolu, A., & Ahmed, I. (2025). An empirical study on automatically detecting AI-generated source code: How far are we? In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)* (pp. 859–871). IEEE. https://doi.org/10.1109/ICSE55347.2025.00064

Shi, Y., Zhang, H., Wan, C., & Gu, X. (2025). Between lines of code: Unraveling the distinct patterns of machine and human programmers. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)* (pp. 1628–1639). IEEE. https://doi.org/10.1109/ICSE55347.2025.00005

Park, S., Jin, H., Cha, J.-W., & Han, Y.-S. (2025). Detecting code paraphrased by large language models using coding style features. *Engineering Applications of Artificial Intelligence, 162*, 112454. https://doi.org/10.1016/j.engappai.2025.112454

Torres, A., Baltes, S., Treude, C., & Wagner, M. (2025). Information-theoretic detection of unusual source code changes. *Empirical Software Engineering, 30*(5), Article 153. https://doi.org/10.1007/s10664-025-10644-y

Orel, D., Azizov, D., Paul, I., Wang, Y., Gurevych, I., & Nakov, P. (2026). AICD Bench: A challenging benchmark for AI-generated code detection. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 6913–6938). Association for Computational Linguistics. https://doi.org/10.18653/v1/2026.eacl-long.325

Herbert, F. (1965). Dune. Chilton Books.

# AI Acknowledgement

AI tools OpenAI (GPT-5.6 Sol) were used in the completion of this work in the following ways:
Brainstorming, Researching background information, Literature scans, Proof reading and equation formatting.

Any content generated underwent meticulous manual review and subsequently received final approval from the author.